

Assignment 5: Advanced Multi-Client Chat Server using TCP

Name: Kaushal Kumar

Roll : CSB24069

1. Objective

Enhance the multi-client TCP chat application developed in Assignment 4 by implementing advanced messaging features, improved client management, persistent chat history, and efficient server-side message routing. The assignment emphasizes concurrent programming, client-server architecture, state management, performance evaluation, and software evolution by extending an existing codebase.

2. System Architecture

The application follows a **client-server architecture**. A central multi-threaded server listens for incoming client connections on TCP port 5000. Each client establishes a TCP connection with the server and communicates by sending messages.

The server maintains information about connected users, including usernames, socket connections, login time, and online status. When a message is received, the server determines whether it is a broadcast message or a private message. Broadcast messages are forwarded to all connected clients, while private messages are routed only to the intended recipient.

A separate thread is created for each connected client, allowing multiple users to communicate simultaneously. The server also records chat activity in a CSV file and provides features such as user listing, statistics reporting, and message history retrieval upon reconnection.

Components:

- **Server:** Manages client connections, message routing, logging, and user state.
 - **Clients:** Send and receive messages through the server.
 - **CSV Logger:** Stores chat history and performance results for analysis.
-

3. Design Improvements over Assignment 4

Several enhancements were implemented compared to Assignment 4. The server now maintains detailed user state information, including usernames, connection status, login time, and socket details. Support for **private messaging** was added through server-side message routing using the `/msg` command. The application also provides a `/list` command to display all currently connected users and a `/stats` command to view server statistics.

A **chat history logging system** was introduced, storing all messages in a CSV file for persistence and analysis. The server also supports a **reconnect feature**, allowing users to view the last few messages after reconnecting. In addition, automated **performance evaluation** was implemented to measure message delay and throughput for different numbers of clients. These improvements make the application more scalable, manageable, and user-friendly than the previous assignment.

4. Program Design

The system is implemented using a **multi-threaded client-server model** over TCP sockets. The server listens for incoming connections and creates a separate thread for each client, enabling multiple users to communicate simultaneously.

When a client connects, the server requests a username and stores the user's information in a client database. The server continuously listens for messages from connected clients. Broadcast messages are delivered to all users, while private messages are routed to a specific recipient using the `/msg` command.

The server maintains user state information, including username, connection details, login time, and online status. All chat activities are recorded in a CSV file (`chat_history.csv`) for persistence and analysis. Additional commands such as `/list` and `/stats` provide information about connected users and server performance.

The client program provides an interactive interface for sending and receiving messages. A dedicated receiving thread ensures that incoming messages can be displayed while the user continues typing. The performance testing module automates message generation and measures metrics such as average delay and throughput for different client loads. This modular design improves maintainability, scalability, and usability.

5. Experimental Setup

The experiment was conducted using **Mininet** on Ubuntu Linux. A single-switch topology with five hosts (**h1**, **h2**, **h3**, **h4**, and **h5**) was created. The server application was executed on **h1**, while the remaining hosts acted as clients.

TCP communication was used over port **5000**. Multiple clients were connected simultaneously to test broadcast messaging, private messaging, user listing, and reconnection functionality. Wireshark was used to capture and analyze network packets during communication.

For performance evaluation, the automated `performance_test.py` script was executed. The script simulated **2, 3, and 4 clients**, with each client sending **50 messages**. Metrics such as average message delivery delay, throughput, and the number of broadcast and private messages were recorded. The results were stored in `performance_results.csv` and visualized using graphs generated by `generate_graphs.py`.

System Configuration:

- Operating System: Ubuntu Linux
- Network Emulator: Mininet
- Programming Language: Python 3
- Transport Protocol: TCP
- Server Port: 5000
- Packet Analyzer: Wireshark
- Test Scenarios: 2, 3, and 4 concurrent clients sending 50 messages each.

6. Results

The multi-client chat application was tested successfully in the Mininet environment. Multiple clients were able to connect to the server simultaneously and exchange messages without communication errors. Broadcast messages were delivered to all connected clients, while private messages were routed only to the intended recipient. The `/list` and `/stats` commands functioned correctly, and chat activity was successfully recorded in `chat_history.csv`.

Performance evaluation was conducted using the automated testing script. Tests were performed with **2, 3, and 4 clients**, where each client sent **50 messages**. The results showed that as the number of clients increased, the server continued to handle communication efficiently, with only a

slight increase in average message delay. Throughput remained stable, demonstrating the effectiveness of the multi-threaded design.

Wireshark packet captures confirmed successful TCP connection establishment, message transmission, private message routing, and connection termination. The generated graphs illustrated the relationship between the number of clients, message delay, and throughput.

Observed Outcomes:

- Successful communication among multiple concurrent clients.
- Correct delivery of broadcast and private messages.
- Accurate maintenance of user state and chat history.
- Stable server performance under increasing client load.
- Successful generation of performance metrics and graphs.

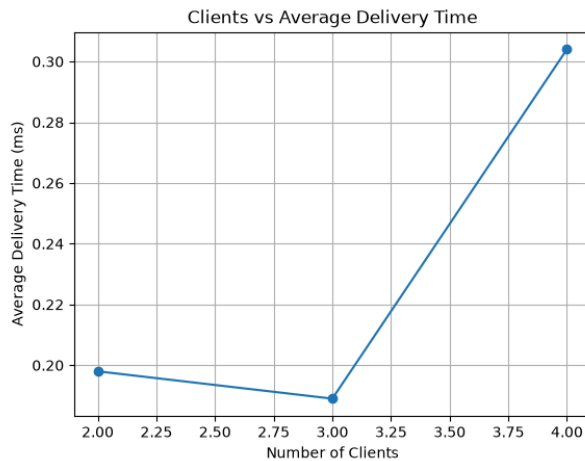
	A	B	C	D	E
1	clients	broadcast_messages	private_messages	avg_delay_ms	throughput_msgs_per_sec
2	2	80	20	0.198	35.193
3	3	120	30	0.189	50.995
4	4	160	40	0.304	65.813
5					
6					
7					
8					
9					

7. Graph Analysis

The generated graphs provide insights into the performance of the multi-client chat application under different client loads.

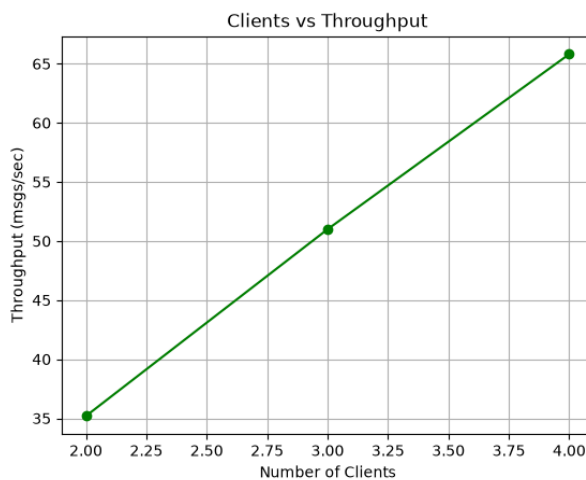
Average Delivery Time:

The "Clients vs Average Delivery Time" graph shows that message delivery time remains low for 2 and 3 clients and increases slightly when the number of clients reaches 4. This increase occurs because the server must handle more simultaneous connections and message transmissions. However, the delay remains very small, indicating efficient server performance.



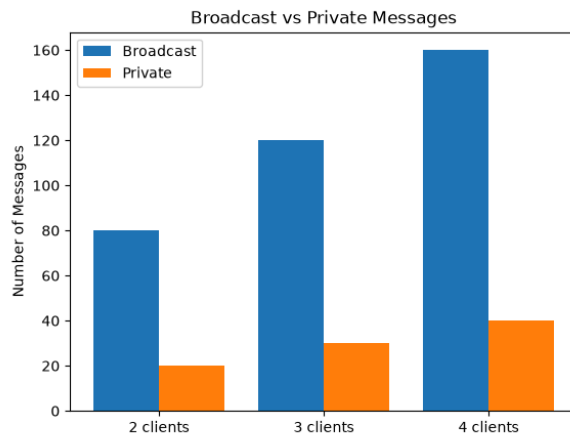
Throughput:

The "Clients vs Throughput" graph shows a steady increase in throughput as the number of clients increases. Throughput rises from approximately 35 messages/second for 2 clients to about 66 messages/second for 4 clients. This demonstrates that the server can process a larger number of messages effectively as client activity increases.



Message Distribution:

The "Broadcast vs Private Messages" graph indicates that broadcast messages are significantly more frequent than private messages in all test cases. This is expected because the test script generates a larger proportion of broadcast messages. As the number of clients increases, both broadcast and private message counts increase proportionally.



Overall Observation:

The results show that the multi-threaded server scales well with increasing client load. Throughput improves as more clients participate, while message delivery delay remains low. These findings demonstrate that the application can efficiently support multiple concurrent users and maintain reliable communication.

8. Wireshark Verification

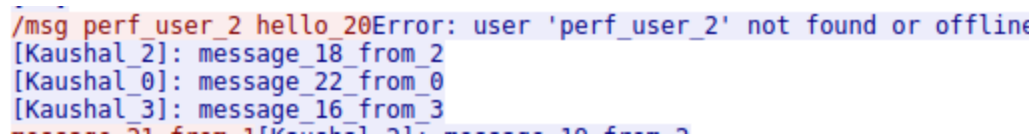
Wireshark was used to verify the network communication between the server and clients. Packet captures were filtered using `tcp.port == 5000` to observe all chat-related traffic. The captured packets confirmed successful TCP connection establishment through the three-way handshake (SYN, SYN-ACK, ACK) when clients connected to the server.

27	96.199...	10.0.0.2	10.0.0.1	TCP	74 49630 → 5000	[SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3023735073 TSecr=0 WS=512
28	96.199...	10.0.0.1	10.0.0.2	TCP	74 5000 → 49630	[SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=3454126393 TSecr=3023735073
29	96.199...	10.0.0.2	10.0.0.1	TCP	66 49630 → 5000	[ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3023735074 TSecr=3454126393

Broadcast message testing showed packets being transmitted from a client to the server and then forwarded to all connected clients.

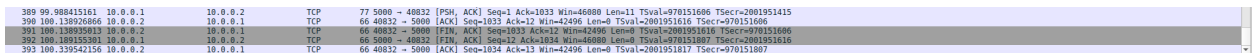
```
message_31_from_1[OK]
[Kaushal_3]: message_27_from_3
[Kaushal_0]: message_33_from_0
[Kaushal_2]: message_29_from_2
message_32_from_1[Kaushal_3]: message_28_from_3
[OK]
[Kaushal_0]: message_34_from_0
message_33_from_1[Kaushal_3]: message_29_from_3
[Kaushal_2]: message_31_from_2
[OK]
message_34_from_1[Kaushal_0]: message_36_from_0
[Kaushal_2]: message_32_from_2
[OK]
```

Private messaging was verified by capturing packets containing the `/msg` command and observing that the server forwarded the message only to the intended recipient.



```
/msg perf user_2 hello 20Error: user 'perf_user_2' not found or offline
[Kaushal_2]: message 18 from 2
[Kaushal_0]: message 22 from 0
[Kaushal_3]: message 16 from 3
```

Client disconnections were also verified through TCP FIN and ACK packets exchanged between the client and server. The packet captures demonstrated reliable message delivery, proper connection management, and correct implementation of both broadcast and private communication features within the application.



389	99.988415161	10.0.0.1	10.0.0.2	TCP	77	5088 → 48832 [PSH, ACK] Seq=1 Ack=1833 Win=46880 Len=11 TSval=970151686 TSecr=2001951415
390	100.139026866	10.0.0.2	10.0.0.1	TCP	66	48832 → 5088 [ACK] Seq=1833 Ack=12 Win=42496 Len=0 TSval=2001951616 TSecr=970151686
391	100.138935813	10.0.0.2	10.0.0.1	TCP	66	48832 → 5088 [FIN, ACK] Seq=1833 Ack=12 Win=42496 Len=0 TSval=2001951616 TSecr=970151686
392	100.139135391	10.0.0.2	10.0.0.2	TCP	66	5088 → 48832 [FIN, ACK] Seq=12 Ack=1834 Win=66880 Len=0 TSval=970151807 TSecr=2001951616
393	100.339542156	10.0.0.2	10.0.0.1	TCP	66	48832 → 5088 [ACK] Seq=1834 Ack=13 Win=42496 Len=0 TSval=2001951817 TSecr=970151807

9. Reflection Answers

1. Why is server-side routing necessary for private messaging?

Server-side routing is necessary because the server keeps track of all connected users and their socket connections. When a user sends a private message, the server identifies the intended recipient and forwards the message only to that user. This ensures privacy and prevents other clients from receiving the message.

2. How does maintaining user state improve the application?

Maintaining user state allows the server to store information such as usernames, connection status, login time, and socket details. This improves user management, enables features like `/list` and reconnect history, and helps the server efficiently route messages to the correct clients.

3. How would the application scale to 100 users?

To support 100 users, the application could use thread pools or asynchronous I/O instead of creating a separate thread for every client. Efficient data structures, load balancing, message queues, and database-backed storage would further improve performance and scalability.

4. Which Wireshark packets verified private messaging?

Private messaging was verified by examining TCP packets exchanged between the client and server on port 5000. The captured packets contained the `/msg` command sent by the sender and the corresponding server-to-recipient transmission, confirming that the message was delivered only to the intended user.

5. What improvements would you implement next?

Future improvements would include message encryption for better security, user authentication with passwords, group chat support, file sharing, persistent message storage using a database, and a graphical user interface (GUI) to improve usability.

10. Conclusion

This assignment successfully implemented a multi-threaded client-server chat application using TCP sockets in Python. The system supports multiple concurrent clients, broadcast messaging, private messaging, user management, chat history logging, and client reconnection. Performance testing was conducted with 2, 3, and 4 clients, demonstrating the server's ability to handle multiple users efficiently. Wireshark analysis confirmed correct packet transmission and private message routing.

Overall, the project enhanced understanding of socket programming, multi-threading, network communication, and performance evaluation in a client-server environment.